

LLMProvider

LLMProvider

Una interfaz, 43 proveedores — con disyuntores, reintentos y monitoreo de salud integrados.

Resumen

LLMProvider es un módulo genérico y reutilizable de Go que define una interfaz unificada LLMProvider junto con los patrones de resiliencia en producción asociados: disyuntor, monitoreo de salud, reintentos con retroceso exponencial y carga diferida. Además, incluye 43 implementaciones concretas de proveedores detrás de ese único contrato, con un adaptador genérico compatible con OpenAI y un sistema de descubrimiento de modelos honesto, sin respaldos codificados.

Descripción breve

Módulo reutilizable de Go que expone una única interfaz LLMProvider (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) junto con primitivas de tolerancia a fallos — disyuntor, monitor de salud, reintentos con retroceso exponencial y jitter, inicialización diferida— sobre 43 adaptadores de proveedores y un adaptador genérico compatible con OpenAI. Seguro para hilos.

Descripción detallada

LLMProvider es la capa de abstracción que todo servicio que consume LLM necesita, pero que casi nadie implementa correctamente: la infraestructura poco glamurosa que distingue una demostración de un sistema capaz de sobrevivir al contacto con tráfico real. Define una única interfaz consciente de capacidades —Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig—, de modo que el código de la aplicación apunta a un solo contrato, sin importar cuál de los 43

backends responda la llamada. Luego, incorpora el refuerzo operativo que convierte llamadas frágiles a proveedores en algo que puede ejecutarse en producción sin contener la respiración.

Un disyuntor de tres estados (cerrado → abierto → semiabierto) envuelve de forma transparente a cualquier proveedor — *incluyendo su canal de streaming*, donde un flujo vacío se cuenta correctamente como fallo—, de modo que un backend con problemas puede activarse y evitar que colapse todo el servicio. Un `CircuitBreakerManager` centralizado rastrea todos los disyuntores a la vez. Un monitor de salud configurable evalúa continuamente a los proveedores en estados de saludable/degradado/no saludable/desconocido mediante comprobaciones basadas en umbrales e intervalos, para detectar la degradación antes de que se convierta en una interrupción. La lógica de reintentos combina retroceso exponencial con jitter, tomando decisiones basadas en el estado: reintenta solo los errores que valen la pena (429, 5xx, fallos transitorios de red), sin malgastar ciclos en errores 4xx o contextos cancelados, y con límites en los retrasos para evitar tormentas de retroceso.

Además, un patrón de inicialización diferida pospone la construcción de cada proveedor hasta su primer uso real, una decisión de diseño deliberada que hace que el registro de los 43 proveedores sea prácticamente gratuito.

El módulo incluye 43 paquetes de proveedores concretos y un adaptador genérico compatible con OpenAI, que implementa la interfaz completa frente a *cualquier* endpoint `/v1/chat/completions` —autenticación Bearer, streaming SSE con manejo correcto de `[DONE]`—, de modo que un proveedor sin paquete dedicado se convierte en ciudadano de primera clase en cuanto se apunta el adaptador a su URL. Las credenciales se resuelven en un único lugar (apikeys, siguiendo una convención estricta de `ApiKey_<Proveedor>`), eliminando de raíz toda una clase de errores del tipo “la clave codificada pasó las pruebas, pero la real nunca se conectó y el producto falló en producción”. El descubrimiento de modelos es deliberadamente —casi obstinadamente— honesto: consulta las APIs de los proveedores en vivo detrás de una caché con TTL, y —por normativa— se eliminó por completo el antiguo nivel de respaldo codificado. Cuando falla el descubrimiento en vivo, `LLMProvider` no devuelve *nada* en lugar de un catálogo obsoleto, de modo que el llamador nunca recibe un ID de modelo que parezca válido pero que no pueda invocarse. Todo esto está diseñado para ser seguro en entornos concurrentes.

Por qué lo creamos

Las llamadas ingenuas a LLM fallan en producción: los proveedores limitan el tráfico, degradan el servicio o se caen, y un backend defectuoso puede arrastrar consigo a todo un servicio.

Los catálogos de modelos se desactualizan, y las listas codificadas a mano proporcionan identificadores que ya no funcionan. LLMProvider centraliza la interfaz, los patrones de resiliencia y el descubrimiento honesto para que cada consumidor herede tolerancia a fallos y veracidad sin esfuerzo.

Por qué es un cambio radical

Reduce “integrar un proveedor de LLM” a un solo movimiento: implementar una única interfaz o simplemente apuntar el adaptador genérico a un endpoint. Luego, envuelve automáticamente y transparentemente ese proveedor en mecanismos de *circuit breaking*, monitoreo de salud y reintentos con *backoff* aleatorio. La resiliencia deja de ser algo que cada equipo reinventa (mal, bajo presión y después del primer fallo) para convertirse en el comportamiento predeterminado de la biblioteca en los 43 backends. La ingeniería de confiabilidad se escribe una vez, se prueba a fondo y se hereda gratuitamente por todos los que la importan.

Qué tiene de innovador

- **Una interfaz única con conciencia de capacidades:** la finalización, el streaming, la salud, las capacidades y la validación de configuración se condensan en un solo contrato que todos los backends cumplen de manera idéntica.
- **Envoltura transparente de *circuit breaker*, incluso para streams.** El interruptor protege el canal de CompleteStream, no solo la solicitud/respuesta, y trata un stream vacío como el fallo que realmente es, con notificaciones a los listeners libres de bloqueos y seguras ante *deadlocks*.
- **43 paquetes de proveedores + un adaptador genérico compatible con OpenAI:** los paquetes dedicados se mantienen ligeros, y cualquier proveedor no listado que hable /v1/chat/completions funciona en el momento en que apuntas el adaptador hacia él.
- **Autoridad única de credenciales (apikey):** solo un lugar lee las variables de entorno ApiKey_<Proveedor>, eliminando estructuralmente la discrepancia “pruebas verdes, producto roto” en lugar de limitarse a advertirla.
- **Descubrimiento honesto de modelos (sin *fallback* codificado):** APIs de proveedores en vivo detrás de una caché con TTL; ante un fallo, devuelve nil, nunca un catálogo obsoleto o fabricado que distribuya IDs no invocables.
- **Inicialización diferida con sync.Once:** la construcción se pospone hasta el primer uso, por lo que registrar los 43 proveedores no cuesta casi nada hasta que realmente se llama a uno.

- **Pila de *Challenge* anti-engaño y multirregional:** un ejecutor real que prueba el comportamiento del *circuit*, la salud y los reintentos en cinco regiones, controlado por pruebas de mutación emparejadas (el código sin mutar debe salir con 0; una mutación inyectada debe forzar la salida con 99), de modo que una suite que pasa garantiza un comportamiento funcional.

Mayores desafíos técnicos y cómo los resolvimos

- **Fallos en cascada de proveedores.** Un backend inestable no debe arrastrar consigo a todo un servicio. Se resolvió con un *circuit breaker* de tres estados (cerrado → abierto → semiabierto) que envuelve transparentemente a cualquier proveedor y su *stream*, se abre ante fallos sostenidos, sondea la recuperación en estado semiabierto y es coordinado centralmente por un *CircuitBreakerManager*.
- **Errores transitorios y límites de tasa.** Se resolvió con un *backoff* exponencial sensible al estado más *jitter*:

$$\min(\text{RetrasoInicial} \cdot \text{Multiplicador}^{(n-1)}, \text{RetrasoMáximo}) \pm \text{jitter}$$
de modo que los reintentos se distribuyen en lugar de sincronizarse en un *thundering herd*. Reintenta exactamente lo que debe reintentarse (429, 500, 502, 503, 504 y errores de red) y rechaza desperdiciar intentos en un contexto cancelado o cualquier otro 4xx.
- **Escalar con muchos proveedores registrados pero no utilizados.** Con 43 proveedores registrados pero solo unos pocos activos en un servicio dado, la construcción ansiosa sería un desperdicio. Se resolvió con inicialización diferida protegida por *sync.Once*, de modo que solo los proveedores que realmente se llaman asumen su costo de configuración.
- **Distribución de IDs de modelos inválidos.** Se resolvió eliminando por completo el nivel de *fallback* de descubrimiento codificado (según CONST-036) y devolviendo nada ante un fallo en el descubrimiento en vivo, además de una copia defensiva al devolver para que un llamador no pueda mutar la caché ni competir con otro lector. La veracidad se impone estructuralmente, no por convención.
- **Streaming + corrección en concurrencia.** El modo de fallo sutil es un *deadlock* entre el bloqueo del *breaker* y sus *callbacks* de listeners. Se resolvió tomando instantáneas de los listeners y notificándolos fuera del bloqueo con un tiempo de espera de 5 segundos, y desbloqueando antes de notificar en el reinicio, con todos los componentes diseñados para uso concurrente y validados por la suite -race.

Pila tecnológica

- **Go (1.25.3)** — seleccionado por su excelente concurrencia, binarios estáticos y una robusta biblioteca estándar; incluye el módulo, la interfaz, todos los primitivos de resiliencia y los 43 adaptadores.
- **net/http (stdlib)** — HTTP sin dependencias externas: impulsa los clientes por proveedor, el adaptador genérico compatible con OpenAI y las llamadas de descubrimiento en vivo, por lo que no hay transporte de terceros que auditar o parchear.
- **logrus** — registro estructurado y con niveles de severidad justo donde los operadores necesitan visibilidad: en las transiciones de estado del cortacircuitos y en la ruta de descubrimiento.
- **testify** — motoriza el conjunto de pruebas y, crucialmente, el anclaje de ramas de mutación que da significado a una ejecución exitosa.
- **yaml.v3** — analiza los paquetes de internacionalización y la configuración en un formato que sigue siendo editable por humanos.
- **digital.vasic.models** — los tipos compartidos LLMRequest / LLMResponse / ProviderCapabilities, mantenidos en un solo lugar para que todos los adaptadores hablen el mismo lenguaje (una dependencia en tiempo de ejecución documentada).
- **Paquetes propios** — circuit, health, retry, apikeys, discovery, providers/ (43 proveedores + generic) e i18n: la superficie de resiliencia e integración dividida en unidades pequeñas e independientes, probables por separado, en lugar de un monolito.
- **.env + ~/api_keys.sh (convención ApiKey_<Proveedor>)** — una única fuente de verdad inequívoca para las credenciales, de modo que las claves se conectan de la misma manera en pruebas y en producción.
- **Suite de pruebas de carrera del Makefile (-race -p 1) + Ejecutor de Desafíos** — la columna vertebral anti-engaños: el detector de carreras demuestra la corrección en concurrencia, y el Ejecutor de Desafíos somete el comportamiento real a caos, DDoS, escalado, estrés, descubrimiento en vivo y escenarios sin suspensión.

Estado y notas de transparencia

- **Estado: beta.** Módulo reutilizable desacoplado; el repositorio GitHub es público.
- **Licencia: por definir.** Inconsistente — doc.go indica MIT, mientras que existe un archivo LICENSE al estilo Apache-2.0 — verificar antes de publicar.
- LLMsVerifier es la fuente única de verdad ascendente para el catálogo canónico de modelos. El manifiesto helix-deps.yaml parece obsoleto (declara deps: [] mientras que la

documentación menciona una dependencia de `digital.vasic.models`); el “Nivel 2 (`models.dev`)” de discovery es un *stub* planificado, no activo.

Nivel de prioridad: Helix-principal (clúster LLM-infraestructura — módulo reutilizable desacoplado). Se ubica después de HelixTrack.